

Chapter 70

Excel, Google Sheets, VBA & BI Question Bank

Answer the spreadsheet, automation, and BI questions that come up across screening calls, live exercises, and case interviews for analyst, BA, BI, and automation-track roles · read a broken macro and fix it live · critique a real dashboard the way a hiring manager would

Approved chapter · v1

Contents

Chapter 70. Excel, Google Sheets, VBA & BI Question Bank

- 70.1 Formulas, lookups, and logic
- 70.2 Pivot tables and data analysis
- 70.3 Power Query
- 70.4 Google Sheets: QUERY, ARRAYFORMULA, and IMPORTRANGE
- 70.5 VBA and Excel macros
- 70.6 Google Apps Script
- 70.7 DAX and Power BI modeling
- Dashboard critiques
- Common mistakes and how to spot them
- In the real world: the interview that turned on one broken macro
- Tools
- The project
- Final-week revision list
- Key terms
- Where this leads

Chapter 70. Excel, Google Sheets, VBA & BI Question Bank

Part VIII — The Interview Playbook

Chapter at a glance

You will learn to: answer the spreadsheet, automation, and BI questions that come up across screening calls, live exercises, and case interviews for analyst, BA, BI, and automation-track roles · read a broken macro and fix it live · critique a real dashboard the way a hiring manager would · walk out with a 20-question final-week revision list.

How this chapter is built, and why it looks different from a normal chapter. Every **core question** gives a one-line answer you can actually recall under pressure, a memory hook ("Remember it as..."), a compact tier table instead of long paragraphs, and the usual follow-ups/red flags/where-to-learn-it. Every **rapid-fire section** is a scan table, not prose: question, one-line answer, one extra-point line, nothing to read twice. Extra-point moves are labeled against Chapter 69's twelve. Numeric results for the formula questions were verified by recalculating a real 6-row Riverstone order table in a spreadsheet engine (companion file `riverstone_orders_mini.md`), not computed by hand alone. VBA, Apps Script, DAX, and Power Query snippets are written from current, well-documented syntax but weren't executed live in this environment; see the status report for what that means for verification.

Learn it in pointers use chapter-level references (Chapters 10, 11, 16, 19) since this chat doesn't have those chapters' approved text to check exact section numbers against: flagged for a re-check once that text is available.

70.1 Formulas, lookups, and logic

Q70-001 · Total revenue for one customer, with a minimum-quantity filter

Remember it as: SUMIFS, sum range first. SUMIF, sum range last. They're backwards from each other on purpose to trip you up.

Answer in one line: `=SUMIFS(net_revenue_range, customer_range, "Sharma Hardware", quantity_range, ">=3")` → **1,894** on the practice table.

Tier	What to say
Passes	Filter manually, or use <code>SUMIF</code> (breaks past one condition)
Strong	<code>SUMIFS</code> handles multiple AND conditions; sum range comes first, unlike <code>SUMIF</code>
Extra points	+ [Validate] cross-check against a pivot filtered the same way + [Scale] use a Table reference (<code>Orders[net_revenue]</code>) so new rows are picked up automatically

Likely follow-ups: Same with COUNTIFS? Different in Google Sheets? "Top customer," not a named one? **Red flag:** manually filtering instead of a formula; nesting SUMIFs for multiple conditions. **Learn it in:** Chapter 10.

Q70-002 • XLOOKUP VS INDEX / MATCH VS VLOOKUP

Remember it as: VLOOKUP only looks right and defaults to "close enough." XLOOKUP looks any direction and defaults to exact.

Answer in one line: VLOOKUP is rightward-only, approximate-match by default; INDEX / MATCH looks any direction; XLOOKUP does INDEX / MATCH's job in one function, exact-match by default.

```
=XLOOKUP("Crate Lid 50L", product_range, unit_price_range) → 90
=INDEX(unit_price_range, MATCH("Crate Lid 50L", product_range, 0)) → 90 (verified)
```

Tier	What to say
Passes	"XLOOKUP is the newer, better version of VLOOKUP"
Strong	+ VLOOKUP breaks if a column is inserted (hard-coded column number) + always pass FALSE for exact match
Extra points	+ [Edge cases] VLOOKUP's silent approximate-match default is the #1 source of wrong-but-plausible results + [Trade-offs] use INDEX / MATCH on files that might open in an older Excel + [Validate] wrap risky lookups in IFERROR

Likely follow-ups: Duplicate lookup values? What does MATCH's 3rd argument do? Can plain VLOOKUP look leftward? **Red flag:** claiming VLOOKUP can look leftward; not knowing the approximate-match default. **Learn it in:** Chapter 11.

Q70-003 • Absolute vs. relative references

Remember it as: \$ = stays put when copied. No \$ = shifts with the copy. F4 cycles through all four combinations.

Answer in one line: B2 shifts when copied (relative); \$B\$2 never moves (absolute); \$B2 / B\$2 lock only column or only row (mixed).

Tier	What to say
Passes	"\$ locks a reference"
Strong	+ a concrete failure example: a tax-rate cell referenced as B2 instead of \$B\$2, copied down a column, silently multiplies by the wrong cell every row, no error thrown
Extra points	+ [Edge cases] copying across and down needs a mixed reference in each direction, not one absolute lock + [Validate] spot-check the last cell after copying, not just the first

Likely follow-ups: What's F4 do? Named Ranges vs. absolute references? Table references? **Red flag:** can't produce a concrete example of what breaks. **Learn it in:** Chapter 10.

Q70-004 • Classify each order into a size tier

Remember it as: Nested IFs: most restrictive condition first, or the wrong tier wins.

Answer in one line: `=IF(I2>1000,"Large",IF(I2>500,"Medium","Small"))` : order matters; check Large before Medium.

Tier	What to say
Passes (broken)	<code>=IF(I2>500,"Medium",IF(I2>1000,"Large","Small"))</code> : never returns "Large," since anything over 1000 is caught by ">500" first
Strong	Fixed order above. Verified: row 2 (net revenue 850) → "Medium"
Extra points	+ [Edge cases] what happens exactly at 1000? > excludes it; confirm with the spec whether the boundary is inclusive + [Trade-offs] past 2-3 tiers, switch to <code>IFS</code> or a lookup table

Likely follow-ups: Same with `IFS` ? With a lookup table? Tier boundary changes next quarter? **Red flag:** ordering conditions least-restrictive first (exactly the broken version above); not testing boundaries. **Learn it in:** Chapter 10.

Q70-005 • `SUMPRODUCT` , and when to reach for it over `SUMIFS`

Remember it as: `SUMPRODUCT` is a dot product: the same $w \cdot x$ from Chapter 35, just in a spreadsheet.

Answer in one line: `=SUMPRODUCT(quantity, unit_price, (1-discount/100))` multiplies arrays element-by-element and sums the result → **7,674** on the practice table, exactly matching `SUM` of the `net_revenue` column.

Tier	What to say
Passes	"It multiplies things and adds them up"
Strong	+ it's array arithmetic, no helper column needed + verified result reconciles to the helper-column total
Extra points	+ [Trade-offs] for a single condition, <code>SUMIFS</code> is more readable; <code>SUMPRODUCT</code> earns its place for weighted totals or OR-across-columns logic <code>SUMIFS</code> can't express + [Edge cases] all arrays must match in size/shape or it errors

Likely follow-ups: Weighted average with `SUMPRODUCT` ? Old-Excel array-formula equivalent (`{ }`)? Is this a dot product in Python? **Red flag:** no mention of it being array arithmetic. **Learn it in:** Chapters 10 and 35.

Rapid-fire, 70.1

#	Question	One-line answer	Extra point
Q70-006	<code>COUNT</code> vs <code>COUNTA</code> vs <code>COUNTBLANK</code> ?	Numeric cells only / any non-blank cell / empty cells	[Edge cases] a formula returning "" looks blank but isn't; <code>COUNTA</code> still counts it
Q70-007	Formula for "days since this order"?	<code>=TODAY()-order_date</code> , format as Number not Date	[Edge cases] if it displays as a date, reformat as General/Number
Q70-008	<code>IFERROR</code> vs <code>IFNA</code> ?	Catches every error type / catches only <code>#N/A</code>	[Business] <code>IFERROR</code> used broadly can hide a real bug behind a friendly blank
Q70-009	<code>FILTER</code> + <code>UNIQUE</code> together?	<code>=UNIQUE(FILTER(customers, category="Storage"))</code> , spills automatically	[Depth] nothing below/beside can already have data, or <code>#SPILL!</code>

#	Question	One-line answer	Extra point
Q70-010	TEXT function, one use?	=TEXT(order_date, "MMMM") → "January" (verified)	[Edge cases] result is now text, can't do date math on it anymore
Q70-011	Named Range, why use one?	A reusable name for a cell/range, e.g. TaxRate for \$B\$2	[Scale] a whole-column Named Range auto-extends as rows are added
Q70-012	Common cause of #REF! ?	A formula references a cell/row/column that's since been deleted	[Validate] use Trace Error / dependent-tracing to find the source, not just the symptom
Q70-013	Flag near-duplicate names (spacing/case)?	=TRIM(LOWER(A2))=TRIM(LOWER(A3))	[Business] silent near-duplicates are a common invisible cause of pivot tables splitting one customer into two rows
Q70-014	Conditional formatting "stop if true"?	Skips later rules once an earlier matching rule fires	[Edge cases] without it, two overlapping rules silently conflict and only the last one visually wins
Q70-015	Table (Ctrl+T) vs. plain range?	Auto-expands formulas/formatting as rows are added; supports structured references	[Scale] building dashboards on Tables, not fixed ranges, is the easiest way to survive new data

70.2 Pivot tables and data analysis

Q70-016 · Pivot grand total doesn't match a manual SUM

Remember it as: Refresh, then filters, then aggregation type: in that order.

Answer in one line: Three usual suspects, checked in order: source range doesn't cover new rows → a hidden filter/slicer is active → the value field silently defaulted to Count instead of Sum (one stray text value in a numeric column does this).

Tier	What to say
Passes	"Check the source range"
Strong	The three-suspect list above, in order
Extra points	+ [Validate] refresh first, then check for active filters, before assuming the data is wrong + [Scale] build the pivot on a Table to eliminate the first cause outright

Likely follow-ups: Change the default summary calculation? Calculated field vs. calculated item? Show values as % of total? **Red flag:** jumping to "the data must be wrong" without checking the pivot's own settings first. **Learn it in:** Chapter 11.

Q70-017 · Revenue by customer by month, with a running total

Remember it as: "Show Values As → Running Total In": the pivot does it natively; don't build it by hand.

Answer in one line: Customer in Rows, month (grouped from the date field) in Columns, net revenue in Values twice, the second one set to "Running Total In" by month.

Tier	What to say
Passes	"Customer in Rows, months in Columns, revenue in Values" (misses the running-total ask)
Strong	+ add the field a second time, "Show Values As → Running Total In"
Extra points	+ [Edge cases] grouping by "Month" alone across years merges Jan 2024 and Jan 2025 unless "Years" is also grouped + [Validate] the running total's last value should match a plain SUM for that customer

Likely follow-ups: Slicer for region? Grouping vs. calculated field? % of the customer's own annual total instead? **Red flag:** not knowing "Show Values As" exists; building the running total with a manual formula. **Learn it in:** Chapter 11.

Q70-018 · Calculated field's aggregation trap

Remember it as: A calculated field runs AFTER the summing, not before. Averaging a ratio this way is almost always wrong.

Answer in one line: A calculated field's formula runs on already-summed totals per cell, not row by row before summing: fine for Revenue/Orders, wrong for averaging something like a discount percentage directly.

Tier	What to say
Passes	"A formula field inside the pivot"
Strong	+ it computes on post-aggregation totals, which breaks for non-additive ratios like an averaged percentage
Extra points	+ [Edge cases] the classic version: AVERAGE of discount % across rows $\neq$ total discount $\div$ total revenue + [Validate] for any ratio-based calculated field, check its grand total against a hand-computed value from the raw totals

Likely follow-ups: Calculated item vs. calculated field? Does DAX have the same trap? (Yes: it's why CALCULATE and context matter, §70.7.) **Red flag:** not knowing calculated fields aggregate post-summary. **Learn it in:** Chapter 11.

Rapid-fire, 70.2

#	Question	One-line answer	Extra point
Q70-019	Slicer vs. filter?	Visual control that can drive multiple pivots/charts at once / affects only its own pivot	[Business] slicers make a report feel interactive to a non-technical viewer
Q70-020	Top 3 customers by revenue in a pivot?	Value Filter → "Top 10," adjusted to 3, on the summed field	[Edge cases] ties at the boundary still only show 3 rows unless handled explicitly
Q70-021	Pivot chart vs. a normal chart on pivot data?	Linked, auto-updates with the pivot / manual rebuild needed if the pivot's shape changes	[Trade-offs] pivot charts are easier to maintain, harder to fully customize
Q70-022	Refresh all pivots at once?	Data → Refresh All; in VBA, ThisWorkbook.RefreshAll	[Scale] set "Refresh on open" so nobody sees stale numbers

#	Question	One-line answer	Extra point
Q70-023	"Item2" appears in row labels: why?	A blank or error-valued source cell got a generic placeholder label instead of being dropped	[Validate] treat it as a data-quality signal, not a display glitch
Q70-024	Each customer's revenue as % of grand total?	Add revenue again → "Show Values As → % of Grand Total"	[Business] fastest way to answer "who matters most" with zero manual math
Q70-025	AVERAGE /pivot average, with blanks in the data?	Blanks are excluded from the denominator, not counted as zero	[Edge cases] text mixed into a numeric column can silently flip the field to Count instead (Q70-016)
Q70-026	"Days between first and most recent order" per customer?	Add order_date to Values twice (Min, Max), calculated field = Max – Min	[Depth] a spreadsheet-native mini version of Chapter 37's churn features

70.3 Power Query

Q70-027 · Combine 12 monthly files into one clean table

Remember it as: "From Folder" once, refresh forever. Copy-paste once, redo forever.

Answer in one line: Power Query → "From Folder" → "Combine & Transform" stacks every file with matching columns into one table; clean once in the query editor, and every step replays automatically on refresh, including against a 13th file dropped into the folder later.

Tier	What to say
Passes	"Copy-paste all 12 files into one sheet" (works once, not repeatable)
Strong	"From Folder" + "Combine & Transform," cleaning steps recorded once
Extra points	+ [Scale] the whole point is next month's refresh needs zero manual work + [Edge cases] select columns by name, not position, so a reordered column in one file doesn't silently misalign + [Validate] check row count against 12 files × their own row counts to catch a silently skipped file

Likely follow-ups: Different column orders across files? A file that fails to load? How does this compare to VBA doing the same thing? **Red flag:** manual copy-paste as the final answer, no mention of repeatability. **Learn it in:** Chapter 11 (compare with Chapter 19's VBA version).

Q70-028 · Power Query "Merge" vs. "Append"

Remember it as: Append = SQL UNION (stack rows). Merge = SQL JOIN (add columns).

Answer in one line: Append stacks tables (same columns, more rows); Merge joins tables side by side on a key (new columns), with the same join types SQL has: Left/Right/Inner/Full Outer/Anti.

Tier	What to say
Passes	"They both combine tables"
Strong	Append = union, same columns; Merge = join, on a matching key

Tier	What to say
Extra points	+ [Depth] Merge's join-type dropdown maps directly onto Chapter 12's SQL join types + [Edge cases] a duplicate-key Merge can silently fan out rows, exactly Chapter 13's fan-out trap + [Validate] check the row count after a Merge against expectations

Likely follow-ups: Anti-join in Power Query? What does "expand" do after a Merge? SQL equivalent?

Red flag: confusing Merge and Append. **Learn it in:** Chapter 11 (and Chapter 12 for the join logic).

Rapid-fire, 70.3

#	Question	One-line answer	Extra point
Q70-029	What's a Power Query "step"?	A named, reorderable transformation recorded in Applied Steps; later steps run on earlier steps' output	[Edge cases] renaming a column, then referencing the old name later, breaks the query
Q70-030	Split "City, State" into two columns?	Split Column → By Delimiter (comma)	[Edge cases] inconsistent commas or extra ones need "By Number of Characters" or a custom <code>Text.Split</code> instead
Q70-031	What does "Unpivot Columns" do?	Wide → long/tall data; Power Query's version of pandas' <code>melt</code>	[Business] raw exports arrive wide (for humans); most analysis wants long, so this is often the first real cleaning step
Q70-032	Change a column's data type: why does it matter?	Right-click header → Change Type; later numeric/date steps fail or misbehave on a column still typed Text	[Validate] check for new "Errors" on rows that didn't convert cleanly
Q70-033	"Close & Load" vs. "Close & Load To"?	Default table output / choose destination (table, pivot connection, connection-only)	[Scale] "Connection only" for staging queries keeps the workbook lean

70.4 Google Sheets: QUERY, ARRAYFORMULA, and IMPORTRANGE

Q70-034 • Total revenue by category, sorted, with QUERY

Remember it as: QUERY is SQL wearing a spreadsheet disguise: column letters instead of names, otherwise the same rules.

Answer in one line:

```
=QUERY(A2:I7, "SELECT D, SUM(I) WHERE D IS NOT NULL GROUP BY D ORDER BY SUM(I) DESC LABEL SUM(I) 'Total Revenue'")
```

GROUP BY is required whenever you aggregate alongside a non-aggregated column: the exact same SQL rule from Chapter 12.

Tier	What to say
Passes	"I'd build a pivot table instead" (dodges the actual QUERY question)
Strong	The formula above, explaining SELECT / GROUP BY / ORDER BY / LABEL

Tier	What to say
Extra points	+ [Depth] columns are referenced by spreadsheet letter, not header name, which trips people up + [Trade-offs] a pivot is faster for a one-off; <code>QUERY</code> wins when the result must feed another formula or chart live

Likely follow-ups: Add a WHERE for a date range? LIMIT equivalent? How's this different from a pivot's output? **Red flag:** not knowing `QUERY` exists at all; confusing column letters with real header names. **Learn it in:** Chapter 11.

Q70-035 · What `ARRAYFORMULA` does, and when it's actually needed

Remember it as: `SUMIFS` already thinks in ranges. Plain `IF` doesn't, until you wrap it.

Answer in one line: `=ARRAYFORMULA(IF(D2:D7="Storage","Yes","No"))` applies a single-cell-style formula to a whole range at once; functions like `SUMIFS` already do this natively and don't need the wrapper.

Tier	What to say
Passes	"It applies a formula to a whole range"
Strong	+ explains <i>why</i> : some formulas naturally operate on ranges, others (plain <code>IF</code> , & concatenation) don't unless wrapped
Extra points	+ [Business] one <code>ARRAYFORMULA</code> with an open-ended range (<code>D2:D</code>) auto-covers new rows: the single most common way a shared sheet quietly breaks (a new row added below the last formula-filled one) + [Edge cases] open-ended ranges can be slow on a very large sheet

Likely follow-ups: Difference from Excel's dynamic arrays? When is `ARRAYFORMULA` around `SUMIFS` redundant? **Red flag:** not knowing why some formulas need it and others don't. **Learn it in:** Chapter 11.

Rapid-fire, 70.4

#	Question	One-line answer	Extra point
Q70-036	<code>IMPORTRANGE</code> , and its setup trap?	Pulls a range from another Sheets file by URL; needs a one-time "Allow access" click	[Edge cases] revoked source access silently breaks every downstream <code>IMPORTRANGE</code> (<code>#REF!</code>)
Q70-037	Pull live external data into Sheets?	<code>GOOGLEFINANCE</code> for currency/stocks; <code>IMPORTXML</code> / <code>IMPORTHTML</code> for public web; Apps Script <code>UrlFetchApp</code> for an authenticated API	[Trade-offs] <code>IMPORT*</code> is simplest but fragile if a source page changes structure
Q70-038	<code>FILTER</code> vs. <code>QUERY</code> 's <code>WHERE</code> clause?	Native boolean filtering, integrates with other array formulas / SQL-like, can filter + aggregate + sort in one formula	[Trade-offs] <code>FILTER</code> alone is more readable; <code>QUERY</code> avoids nesting two formulas for filter-plus-aggregate
Q70-039	Protect a shared sheet from accidental overwrites?	Data → Protected sheets and ranges, restricting edit access to formula cells	[Business] the cheapest fix for the most common cause of "why are the numbers wrong" on a team sheet

70.5 VBA and Excel macros

Q70-040 · Find the last used row

Remember it as: Start from the bottom and go up. Starting from the top and going down stops at the first gap.

Answer in one line:

```
lastRow = Cells(Rows.Count, "A").End(xlUp).Row
```

`.End(xlDown)` from row 1 stops at the first blank cell, silently returning too small a number if there's any gap; going up from the very last row doesn't have that problem.

Tier	What to say
Passes (broken)	<code>Range("A1").End(xlDown).Row</code> : fails on any gap in the column
Strong	The bottom-up version above
Extra points	+ [Scale] <code>Rows.Count</code> works regardless of file format or sheet size + [Validate] <code>Debug.Print lastRow</code> against a small known test file before trusting it

Likely follow-ups: Last used column instead? Data that might exist below an apparent gap? Same thing in Python? **Red flag:** using the top-down version as the final answer with no acknowledgment of the gap problem. **Learn it in:** Chapter 19.

Q70-041 · Fix the broken macro (infinite loop)

Remember it as: A `Do While` loop with no increment is a promise to run forever.

```
' BROKEN: never terminates
Sub MarkLargeOrders()
    Dim i As Integer
    i = 2
    Do While Cells(i, 1).Value <> ""
        If Cells(i, 9).Value > 1000 Then
            Cells(i, 10).Value = "Large"
        End If
    Loop
End Sub
```

What's wrong: `i` never increments, so `Cells(i, 1)` never changes and the loop condition never goes false.

```
' FIXED
Sub MarkLargeOrders()
    Dim i As Long, lastRow As Long
    lastRow = Cells(Rows.Count, "A").End(xlUp).Row
    For i = 2 To lastRow
        If Cells(i, 9).Value > 1000 Then Cells(i, 10).Value = "Large"
    Next i
End Sub
```

Tier	What to say
Diagnosis	Missing increment inside the <code>Do While</code> loop
Fix	Bounded <code>For...Next</code> loop instead, with <code>lastRow</code> pre-computed
Extra points	+ [Edge cases] also switched <code>Integer</code> → <code>Long</code> (<code>Integer</code> caps at 32,767, a real silent-failure risk) + [Scale] cell-by-cell is slow at scale; read into an array, loop in memory, write back once + [Validate] test against the 6-row practice table first, where the answer is known by hand

Likely follow-ups: Speed this up on 50,000 rows? Array-based version? What if column I is blank instead of a number? **Red flag:** fixing the infinite loop but missing the `Integer` overflow risk. **Learn it in:** Chapter 19.

Q70-042 · The VBA object model

Remember it as: *Application* → *Workbooks* → *Worksheets* → *Range*. Biggest to smallest, like folders inside folders.

Answer in one line: `Application` (Excel itself) → `Workbooks` (every open file) → one `Workbook` → `Worksheets` → one `Worksheet` → `Range`. Full form:

`Application.Workbooks("Sales.xlsx").Worksheets("Orders").Range("A1")` ; VBA lets you drop the unambiguous parts.

Tier	What to say
Passes	"They're all objects in VBA"
Strong	The nesting hierarchy above, with a fully-qualified example
Extra points	+ [Edge cases] relying on "the active sheet" implicitly is a common source of a macro that works manually but breaks when triggered from a button on a different tab + [Scale] for any reusable macro, always qualify <code>Range / Cells</code> with an explicit <code>Worksheet</code>

Likely follow-ups: `ActiveSheet` vs. `ThisWorkbook.Sheets(1)` ? What does `With` do? Referencing a closed workbook? **Red flag:** relying entirely on implicit active-sheet/active-workbook behavior. **Learn it in:** Chapter 19.

Q70-043 · Making a VBA macro run faster

Remember it as: *Screen updating off, calculation manual, arrays not cells: in that order of impact.*

Answer in one line: `Application.ScreenUpdating = False` + `Application.Calculation = xlCalculationManual` at the start (restore both at the end); read a range into an array, loop in memory, write back in one block; avoid `Select / Activate` entirely.

Tier	What to say
Passes	"Avoid unnecessary loops" (no concrete technique)
Strong	The three changes above, in order of typical impact
Extra points	+ [Scale] array-based rewrite is often a 10-100x speedup on tens of thousands of rows, not marginal + [Validate] time it with <code>Timer</code> before/after each change, measured not assumed + [Maintenance] restore screen-updating/calculation mode even on an error path, or a crash leaves the workbook stuck

Likely follow-ups: Show the array read-loop-write pattern. Risk of forgetting to restore `ScreenUpdating` ? Python/pandas comparison? **Red flag:** no mention of screen updating or manual calculation mode. **Learn it in:** Chapter 19.

Rapid-fire, 70.5

#	Question	One-line answer	Extra point
Q70-044	Personal Macro Workbook: why use it?	Hidden <code>PERSONAL.XLSB</code> that opens with Excel, so a macro runs from any file	[Business] for macros used across many reports, not tied to one workbook
Q70-045	<code>On Error Resume Next</code> : risk?	Skips a line that errors and keeps going	[Edge cases] used too broadly, it silently swallows real bugs too; scope tightly, pair with <code>On Error GoTo 0</code> right after
Q70-046	Custom worksheet function (UDF): limits?	<code>Function</code> instead of <code>Sub</code> , returns a value into the calling cell	[Trade-offs] can't modify other cells or formatting; anything beyond pure calculation needs a <code>Sub</code> instead
Q70-047	Workbook-level vs. Worksheet-level events?	Whole-file events (<code>Open</code> , <code>BeforeSave</code>) / one-sheet events (<code>Change</code> , <code>SelectionChange</code>)	[Edge cases] <code>Worksheet_Change</code> code that itself writes to that sheet can recurse infinitely without <code>Application.EnableEvents = False</code>
Q70-048	Consolidate every sheet into one summary?	Loop <code>ThisWorkbook.Worksheets</code> , skip the summary sheet by name, append each used range	[Scale] loop the whole collection and exclude by name, not by index: sheets get renamed/reordered
Q70-049	UserForm: when to build one?	Custom dialog for structured, validated input from a non-technical user	[Business] trades build time for far fewer "typed the date in the wrong cell" support requests
Q70-050	Debug a VBA macro step by step?	Breakpoint (F9) → Run (F5) → Step (F8), watch the Locals window or <code>Debug.Print</code>	[Real evidence] the VBA equivalent of a Python debugger; interviewers want this workflow described concretely
Q70-051	Save active sheet as PDF and email via Outlook?	<code>ActiveSheet.ExportAsFixedFormat Type:=xlTypePDF</code> then <code>CreateObject("Outlook.Application")</code> → build a <code>MailItem</code> → attach → <code>.Send</code>	[Business] this exact pattern is one of the highest-value real-job automations, and Chapter 19's own project

70.6 Google Apps Script

Q70-052 · Read a range and write results back, without looping cell-by-cell

Remember it as: Every `getValue()/setValue()` is a network trip. Batch it, or the clock (and the quota) runs out.

```
// SLOW: a round trip per cell, risks the execution-time quota
function markLarge() {
  var sheet = SpreadsheetApp.getActiveSheet();
  for (var i = 2; i <= sheet.getLastRow(); i++) {
    if (sheet.getRange(i, 9).getValue() > 1000) sheet.getRange(i, 10).setValue("Large");
  }
}

// FAST: one read, in-memory processing, one write
function markLarge() {
  var sheet = SpreadsheetApp.getActiveSheet();
  var lastRow = sheet.getLastRow();
  var data = sheet.getRange(2, 9, lastRow - 1, 1).getValues();
  var results = data.map(function(row) { return [row[0] > 1000 ? "Large" : ""]; });
  sheet.getRange(2, 10, results.length, 1).setValues(results);
}
```

Tier	What to say
Passes	The slow version: correct, but risks the execution-time quota on a large sheet
Strong	The batch version: one <code>getValues()</code> , process in memory, one <code>setValues()</code>
Extra points	+ [Scale] the direct Apps Script equivalent of Q70-043's VBA array pattern, same reason: minimize round trips + [Edge cases] <code>getValues()</code> returns a 2D array even for one column: <code>row[0]</code> , not <code>row</code> + [Validate] test on the small practice table first

Likely follow-ups: What's the execution time limit, and what if a task genuinely needs longer? Trigger this nightly? **Red flag:** cell-by-cell reads/writes with no acknowledgment of the quota risk. **Learn it in:** Chapter 19.

Q70-053 · Fix the broken script (duplicate emails on every trigger run)

Remember it as: No "already sent" flag means every run re-sends to everyone, forever.

```
// BROKEN: no memory of what's already been sent
function sendConfirmations() {
  var data = SpreadsheetApp.getActiveSheet().getDataRange().getValues();
  for (var i = 1; i < data.length; i++) {
    MailApp.sendEmail(data[i][2], "Confirmed", "Thanks for your order!");
  }
}
```

What's wrong: nothing tracks which rows were already processed, so every trigger firing re-emails every row in the sheet.

```
// FIXED: a per-row "Sent" flag, checked and set
function sendConfirmations() {
  var sheet = SpreadsheetApp.getActiveSheet();
  var data = sheet.getDataRange().getValues();
  var statusCol = 4;
  for (var i = 1; i < data.length; i++) {
    if (data[i][statusCol - 1] === "Sent") continue;
    MailApp.sendEmail(data[i][2], "Confirmed", "Thanks for your order!");
    sheet.getRange(i + 1, statusCol).setValue("Sent");
  }
}
```

Tier	What to say
Diagnosis	No tracking of already-processed rows
Fix	A "Sent" status flag, checked before sending and set after
Extra points	+ [Edge cases] also makes the script safe to manually re-run after a partial failure + [Validate] run it twice on the same test data; the second run should send zero emails + [Business] duplicate customer-facing emails are the kind of automation bug customers actually notice

Likely follow-ups: Set up the time-driven trigger itself? What happens on a partial failure mid-run?

MailApp vs **GmailApp** here? **Red flag:** not identifying the missing "already processed" tracking as the root cause. **Learn it in:** Chapter 19.

Rapid-fire, 70.6

#	Question	One-line answer	Extra point
Q70-054	MailApp vs. GmailApp?	Simple, send-only / can also read, search, label, sends as the user's own Gmail	[Trade-offs] MailApp is enough for simple outbound notifications
Q70-055	Time-driven vs. "on edit" trigger?	Runs on a schedule / fires when a user edits a cell	[Edge cases] simple built-in onEdit(e) can't send email; needs an <i>installable</i> trigger for full permissions
Q70-056	Call an external API from Apps Script?	<code>UrlFetchApp.fetch(url, options)</code> , parse with <code>JSON.parse()</code>	[Business] the mechanism behind syncing a Sheet with a CRM with no backend (Chapter 78 goes deeper)
Q70-057	Execution quota: what if a task exceeds it?	~6 min/run limit; checkpoint progress to a property, then re-trigger to continue	[Scale] the same "checkpoint and resume" idea as Part V's pipelines, smaller scale
Q70-058	<code>SpreadsheetApp.flush()</code> : when needed?	Forces pending writes immediately, instead of batching until the script ends	[Edge cases] without it, reading a cell right after writing it can return the stale, pre-write value

70.7 DAX and Power BI modeling

Q70-059 · Calculated column vs. measure

Remember it as: Column = stored, one value per row, computed once at refresh. Measure = computed live, changes with whatever's filtered.

Answer in one line: A calculated column is computed row-by-row and physically stored; a measure computes on the fly, responsive to whatever filter context (a slicer, a date range) is currently applied.

Tier	What to say
Passes	"They're both formulas in Power BI"
Strong	+ net revenue per order (row-level, stored) is a column; total/average revenue across whatever's filtered is a measure

Tier	What to say
Extra points	+ [Trade-offs] row-only logic → column; anything aggregating or responding to filters → measure, columns can't do that responsively + [Scale] calculated columns bloat memory on large tables since every row stores a copy; measures store nothing + [Business] a measure automatically respects whatever a viewer has filtered on, with zero extra work

Likely follow-ups: Filter context vs. row context? What does `CALCULATE` actually do? Ever need both for the same value? **Red flag:** treating them as interchangeable; not knowing the storage difference. **Learn it in:** Chapter 16.

Q70-060 · Revenue this month vs. same month last year

Remember it as: `CALCULATE` changes the filter context. Everything else in DAX time intelligence is `CALCULATE` wearing a costume.

```
Revenue LY = CALCULATE([Total Revenue], SAMEPERIODLASTYEAR('Date'[Date]))
Revenue YoY % = DIVIDE([Total Revenue] - [Revenue LY], [Revenue LY])
```

Tier	What to say
Passes	"You'd filter to last year and compare" (English, no DAX)
Strong	The two measures above, explaining <code>CALCULATE</code> modifies filter context and needs a proper marked Date table
Extra points	+ [Edge cases] <code>DIVIDE</code> , not <code>/</code> , handles a zero/blank denominator gracefully: matters here since "Revenue LY" can legitimately be blank for a new product + [Depth] <code>CALCULATE</code> underlies almost every time-intelligence function; understand it and the rest follows + [Validate] check one manually-known month against the DAX result before trusting the whole model

Likely follow-ups: `SAMEPERIODLASTYEAR` vs. `DATEADD`? What does a Date table need to work correctly? What's `ALL` inside `CALCULATE`? **Red flag:** using `/` instead of `DIVIDE`; no mention of the Date table requirement; can't explain what `CALCULATE` fundamentally does. **Learn it in:** Chapter 16.

Q70-061 · Star schema, and why it beats one flat table

Remember it as: One fact table (events), several dimension tables (who/what/when) around it, like a star.

Answer in one line: A central fact table (one row per event, holding measures and foreign keys) surrounded by dimension tables (Customer, Product, Date) in one-to-many relationships: smaller, faster for DAX, and gives one unambiguous Date table for time intelligence.

Tier	What to say
Passes	"It's a way of organizing tables with relationships"
Strong	The fact/dimension structure above, with why it beats a flat table
Extra points	+ [Depth] a flat table forces every attribute to repeat on every fact row, bloating memory and confusing filter propagation + [Trade-offs] for a very small dataset, flat can genuinely be fine: the star's benefits compound as data and time-intelligence needs grow + [Real evidence] report load times dropping noticeably purely from a schema change, no measures rewritten

Likely follow-ups: Snowflake schema, when to use it? Many-to-many relationships? Bridge table? **Red flag:** no mention of fact vs. dimension specifically; claiming flat is always simpler. **Learn it in:** Chapter 16 (and Part V for the warehouse version of the same idea).

Q70-062 · Row-level security (RLS): each rep sees only their own accounts

Remember it as: RLS is enforced server-side, per logged-in user: not a filter you apply before publishing.

Answer in one line: Define a role with a DAX filter comparing a `SalesRep` column to `USERNAME()`, under Modeling → Manage Roles; once published, every user sees the report pre-filtered to their own identity automatically.

Tier	What to say
Passes	"You'd filter the data by rep before publishing" (that's not RLS at all)
Strong	The role-and-DAX-filter mechanism above, enforced dynamically per user login
Extra points	+ [Edge cases] test with "View As Role" in Desktop, and a real test account after publishing: RLS can behave unexpectedly with existing report filters or many-to-many relationships + [Scale] dynamic RLS (comparing a column to the logged-in user directly) scales to any headcount with zero per-person maintenance; static RLS doesn't + [Business] one published report, safely different data per viewer, instead of N maintained copies

Likely follow-ups: Static vs. dynamic RLS? Does RLS work in an embedded report? How would you test it before rollout? **Red flag:** confusing RLS with a plain report-level filter. **Learn it in:** Chapter 16.

Rapid-fire, 70.7

#	Question	One-line answer	Extra point
Q70-063	Import mode vs. DirectQuery?	Full copy loaded in-memory, fast, as fresh as last refresh / live query every interaction, always current, slower	[Trade-offs] default to Import; DirectQuery only when near-real-time genuinely matters more than speed
Q70-064	What does <code>ALL</code> do inside <code>CALCULATE</code> ?	Removes filters from a table/column so a measure can compute the unfiltered total	[Edge cases] <code>ALL</code> on a whole table removes every filter from it; <code>ALLEXCEPT / ALL(table[col])</code> is often safer
Q70-065	Power BI dashboard vs. report?	Multi-page, fully interactive, built in Desktop / single-page pinned visuals from multiple reports, built in the Service	[Business] an exec wants a dashboard glance; an analyst wants the underlying report to dig
Q70-066	Handle a many-to-many relationship in the model?	Mark it explicitly many-to-many, or introduce a bridge table for two clean one-to-many relationships	[Depth] a bridge table is usually more predictable than native many-to-many filter propagation
Q70-067	Power Query vs. DAX: where does a transform belong?	Reshape/clean <i>before</i> load, same for every user / calculate <i>within</i> the loaded model, responsive to filters	[Trade-offs] same-every-time → Power Query; responds to what's currently selected → DAX
Q70-068	Translate a Power BI star schema + measures to Tableau?	The schema concept transfers directly; DAX measures move into Calculated Fields; built-in time intelligence (<code>SAMEPERIODLASTYEAR</code>) needs more manual date-function work	[Depth] the modeling concept is portable; the measure syntax and built-in time intelligence are what differ most

Dashboard critiques

Q70-069 · Critique and rebuild: the "everything on one page, no filters" dashboard

The dashboard, as described: One page. Two same-sized big numbers (this year's revenue, last year's, no visual distinction). A 14-slice pie chart of revenue by SKU. A 3,000-row raw table below that. No filters, no date range: always shows all-time data.

What's wrong, fast:

Problem	Why it matters	Fix
Two identical-looking "big numbers"	Forces re-reading both labels every time to tell them apart	One large current number + a small "+X% vs. LY" indicator beside it
14-slice pie chart	Human perception can't reliably compare more than ~5-6 slices by angle	Sorted horizontal bar chart, top 8-10 + "Other"
3,000-row table on the dashboard	A data dump, not a dashboard; buries the summary in scrolling	Drill-through or export, not displayed by default
No filters, no date range	Can never answer "how's this quarter" without a rebuild	A date range slicer driving every visual on the page

Extra-points moves demonstrated: **[Structure]** organized by problem, not stream of consciousness. **[Business]** every fix ties to what the viewer actually needs to do. **[Edge cases]** the "Other" bucket and drill-through both anticipate the data growing past what fits on one screen.

Learn it in: Chapter 16 (and Chapter 5's visualization principles, pending that chapter's confirmed section numbers).

Q70-070 · Critique and rebuild: the "six gauges and a 3D pie, live on production" dashboard

The dashboard, as described: Six gauge charts across the top, one per region, showing progress-to-target %. A 3D exploded pie chart of revenue by sales rep below. Every chart uses a different, unrelated color palette. Refreshes live via DirectQuery against the production database.

What's wrong, fast:

Problem	Why it matters	Fix
Six gauge charts	Visually heavy for one number each; harder to compare at a glance than bars of equal height	Simple KPI cards or one small bar chart
3D exploded pie	3D perspective makes back slices look smaller than they really are, independent of actual value; "exploded" makes it worse	A plain 2D sorted bar chart
Inconsistent color palettes per chart	Forces the eye to re-learn color meaning on every chart	One shared palette (same region, same color, everywhere on the page)
Live DirectQuery for a daily-check metric	Risks slowing production under normal load for no real freshness benefit	Import mode, scheduled refresh

Extra-points moves demonstrated: **[Depth]** naming the *specific, known* perceptual problem with 3D/exploded pies, not just "pie charts are bad." **[Scale]** connecting a visualization choice to a real production-system risk. **[Business]** every fix asks what the viewer actually needs, not "looks unprofessional."

Learn it in: Chapter 16.

Common mistakes and how to spot them

Mistake	Symptom	Fix
<code>VLOOKUP</code> with match-type omitted	Silent wrong results from approximate matching	Pass <code>FALSE</code> explicitly, or use <code>XLOOKUP</code> / <code>INDEX - MATCH</code>
Nested <code>IF</code> s in the wrong order	A middle tier "wins" over a higher one that should fire first	Order most-to-least restrictive; <code>IFS</code> past two levels
Pivot source range excludes new rows	Grand total quietly misses recent data	Build pivots on Tables, not fixed ranges
VBA looping cell-by-cell on a large range	Macro is slow or times out	Array read → in-memory process → array write
Apps Script looping <code>getValue()</code> / <code>setValue()</code>	Script hits its execution quota	Batch with <code>getValues()</code> / <code>setValues()</code>
DAX calculated column used where a measure was needed	Value doesn't respond to report filters	Recompute as a measure if it aggregates or responds to filter context
Dashboard with no date filter	Can't answer "how's this quarter" without a rebuild	Add a date range slicer driving every visual
A 3D or heavily-decorated chart for simple data	Distorts or obscures the comparison it's meant to show	Default to the plainest chart type that shows the pattern

In the real world: the interview that turned on one broken macro

Arjun, a BA candidate, gets a live-coding round: a VBA macro meant to consolidate three regional files, throwing "Subscript out of range." He's never seen this exact error before, and says so out loud, then reasons through it: "This usually means I'm referencing something, a sheet name, an array index, that doesn't exist. Let me check what the code's asking for against what's actually there." The code references `Worksheets("Region2")` ; the actual sheet is named "Region 2," with a space.

He fixes the typo, and then adds one more line, unprompted: a message box reporting how many rows were pulled from each file, so a future user notices immediately if a file silently fails to load. The interviewer's notes, later shared with him, call out that line specifically: "diagnosed a real error methodically, then thought one step past 'it works now.'"

The lesson: a broken-macro round isn't testing whether you already know that specific error message. It's testing whether you have a *method* for one you've never seen, and whether you stop at "it runs" or keep going to "it runs, and it'll tell someone if it silently fails next time."

Tools

Excel and Google Sheets (both free at a basic level). **Power BI Desktop** (free to build; publishing has free/paid tiers). **VBA** ships inside Excel; **Google Apps Script** ships inside every Sheet (Extensions → Apps Script). Nothing beyond what Chapters 10, 11, 16, and 19 already covered.

The project

Goal: one worked example per major topic in this chapter, on your own data or Riverstone's.

1. Write three `SUMIFS` / `COUNTIFS` formulas answering real questions, validated against a pivot table built the slow way.
 2. Build one Power Query combining multiple files (or simulate with multiple sheets), documenting every step's purpose in its name.
 3. Write one VBA macro or Apps Script function that would've taken over ten minutes by hand; time the before-and-after.
 4. Build one Power BI or equivalent dashboard, then critique it yourself using this chapter's table format, honestly, before anyone else does.
 5. Write one DAX measure (or Sheets/Tableau equivalent) that responds correctly to a filter, and prove it by changing the filter.
-

Final-week revision list

Q70-001, Q70-002, Q70-003, Q70-005, Q70-016, Q70-018, Q70-027, Q70-028, Q70-034, Q70-035, Q70-040, Q70-041, Q70-043, Q70-052, Q70-053, Q70-059, Q70-060, Q70-061, Q70-062, Q70-069.

Key terms

ATS-safe formatting · `SUMIFS` / `COUNTIFS` · `XLOOKUP` · `INDEX` / `MATCH` · absolute vs. relative reference · `SUMPRODUCT` · dynamic array (`FILTER` , `UNIQUE`) · pivot table · calculated field vs. calculated column · Power Query · Merge vs. Append · `QUERY` (Sheets) · `ARRAYFORMULA` · `IMPORTRANGE` · VBA object model ·

Personal Macro Workbook · On Error · UserForm · Apps Script quota · `getValues()` / `setValues()` · batching · installable trigger · DAX measure vs. calculated column · `CALCULATE` · filter context · star schema · row-level security (RLS) · Import mode vs. DirectQuery

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, picks up exactly where §70.3's Merge/Append and §70.4's `QUERY` leave off: the same join and aggregation logic, in real SQL.
- **Chapter 78, Automation & Integration Question Bank**, goes further into API integration and scheduling than §70.6's Apps Script coverage does here.
- **Chapters 10, 11, 16, and 19** teach every technique this bank draws on, in full; this chapter tests it, it doesn't re-teach it from scratch.